Introduction

Welcome! This is a book about multi-robot systems. Why? Because it's the future!

Robots are becoming more affordable, more capable, and more useful in many "real life" scenarios. As a result, we are seeing more and more robots that need to share spaces and work together to accomplish tasks. In this book, we will introduce the Robot Operating System 2 (ROS 2) as well as the Robot Middleware Framework (RMF), which is built on ROS 2 and tries to simplify the creation and operation of complex multi-robot systems.

This chapter describes the motivation and goals for ROS 2 and the RMF system for integrating multiple robots.

ROS 2

The Robot Operating System (ROS) is a set of software libraries and tools for building robot applications. From drivers to state-of-the-art algorithms, and with powerful developer tools, ROS has what you need for your next robotics project. And it’s all open source.

Since ROS was started in 2007, a lot has changed in the robotics and ROS community. ROS 1, originally just "ROS", began life as the development environment for the Willow Garage PR2 robot, a high-performance mobile manipulation platform intended for advanced research and development. The original goal of ROS was to provide the software tools users would need to undertake novel research and development projects with this robot. At the same time, the ROS 1 development team knew the PR2 would not be the only robot in the world, nor the most important, so they wanted ROS 1 to be useful on other robots, too. The original focus was on defining levels of abstraction (usually through message interfaces) that would allow much of the software to be reused elsewhere.

ROS 1 satisfied the PR2 use case, but also became useful on a surprisingly wide variety of robots. This included robots similar to the PR2, but also wheeled robots of all sizes, legged humanoids, industrial arms, outdoor ground vehicles (including self-driving cars), aerial vehicles, surface vehicles, and more. ROS 1 adoption also took a surprising turn, happening in domains beyond the mostly academic research community that was the initial focus. ROS-1-based products were coming to market, including manufacturing robots, agricultural robots, commercial cleaning robots, and others. Government agencies were also looking more closely at ROS for use in their fielded systems; NASA, for example, expected to run ROS on the Robonaut 2 deployed to the International Space Station.

All of these applications certainly grew the ROS platform in unexpected ways. Though it held up well, the ROS 1 team believed they could better meet the needs of the broader ROS community

by tackling their new use cases head-on. And so, ROS 2 was born.

The initial goal of the ROS 2 project was to adapt to the changing landscape, leveraging what was great about ROS 1 and improving what wasn't. But there was also a desire to preserve ROS 1 as it existed, to continue working and remain unaffected by the development of ROS 2. So, ROS 2 was built as a parallel set of packages that can be installed alongside and interoperate with ROS 1 (for example, through message bridges).

At the time of writing, we have reached the 13th and last official ROS 1 release, Noetic Ninjemys, and the first LTS release of ROS 2, Foxy Fitzroy.

A large and growing amount of ROS 2 resources can be found on the web. A great place to start is on the ROS Index page for ROS 2 and further along in this book in the ROS 2 chapter.

Enjoy your journey!

Robotics Middleware Framework (RMF)

For a moment, think of any large building. It could be a shopping mall, housing complex, university building, workplace, airport, hospital, hotel, and so on. Are items delivered within the building? Is the building floor cleaned regularly? For most buildings, the answer to both questions is "yes."

Now, let's think of what happens when robots start to perform those tasks. In today's robot marketplace, you can purchase excellent delivery robots, as well as excellent floor-cleaning robots. However, what if the floor is being cleaned at the same time that items are being delivered in the building? This situation is trivial when humans are performing the cleaning and delivery tasks: a quick glance between a delivery person pushing a cart and a custodian cleaning the floor is all it takes to quickly reach a compromise. One or both people will find a way to slightly alter the timing of their task to allow both tasks to be completed.

Unfortunately, robots are nowhere near as capable as humans at abstract reasoning, planning, and informal communication! This type of scenario is what the Robotics Middleware Framework (RMF) tries to help avoid. In today's marketplace, if all robots are purchased from the same manufacturer, the robots in such a single-vendor system will know of each other's existence and will avoid conflicting with each other. However, multi-vendor, multi-robot systems remain an open problem, and we expect that multi-vendor robot deployments will be the norm in all large buildings in the future. To address this situation, RMF provides a set of conventions, tools, and software implementations to allow multiple fleets of robots to interoperate with each other and with shared building infrastructure, such as lifts, doors, corridors, and other natural "bottlenecks" to traffic flows and tasks.

Without a framework for multi-vendor robotics in place, there can be significant but hidden risks for building operators and end users when they are forced to commit to a single system or platform provider. Hidden risks are likely to force an end user to limit their selection of future solutions from that a single provider to minimize operational risk and avoid redundant integration costs. As the scope and scale of robotic deployments increase, this problem is exacerbated, leaving the customer with the perception that there are no good options except to stay with their current provider, and preventing the use of robots from newer entrants to the marketplace.

Beyond the increased cost risk of scaling deployment with different providers, there is also the inherent conflict over shared resources such as lifts, doorways, corridors, network bandwidth, chargers, operations-center screen “real estate,” and human resources such as IT personnel and maintenance technicians. As robotic scale increases, it becomes more cumbersome for an operations team to consider managing a large, heterogeneous, multi-vendor robot environment.

These problem statements were the foundational motivations for the development of RMF.

In the previous "cleaning and delivery" scenario, RMF can act as a traffic controller to help the delivery robot and cleaning robot negotiate a way for both tasks to be accomplished, depending on the relative priority and importance of each task. If the cleaning task is urgent (perhaps a spill occurred in a busy corridor), RMF could route the delivery task through a different set of corridors. If the delivery task is time-critical, RMF could direct the cleaning robot to pause its work and move out of the way until the delivery robot clears the corridor. Of course, these solutions are obvious and could be easily hand-written for this particular "cleaning and delivery" corridor-sharing scenario. The challenge comes from trying to be generic across many scenarios, while also trying to be "future proof" to allow expansion to currently-unknown robots, applications, and task domains.

The rest of the book will dive into these details to show how RMF tries to foresee and prevent resource conflicts and improve the efficiency of multi-vendor, multi-robot systems. There is no magic here! All of the implementations are open-source and available for inspection and customization.

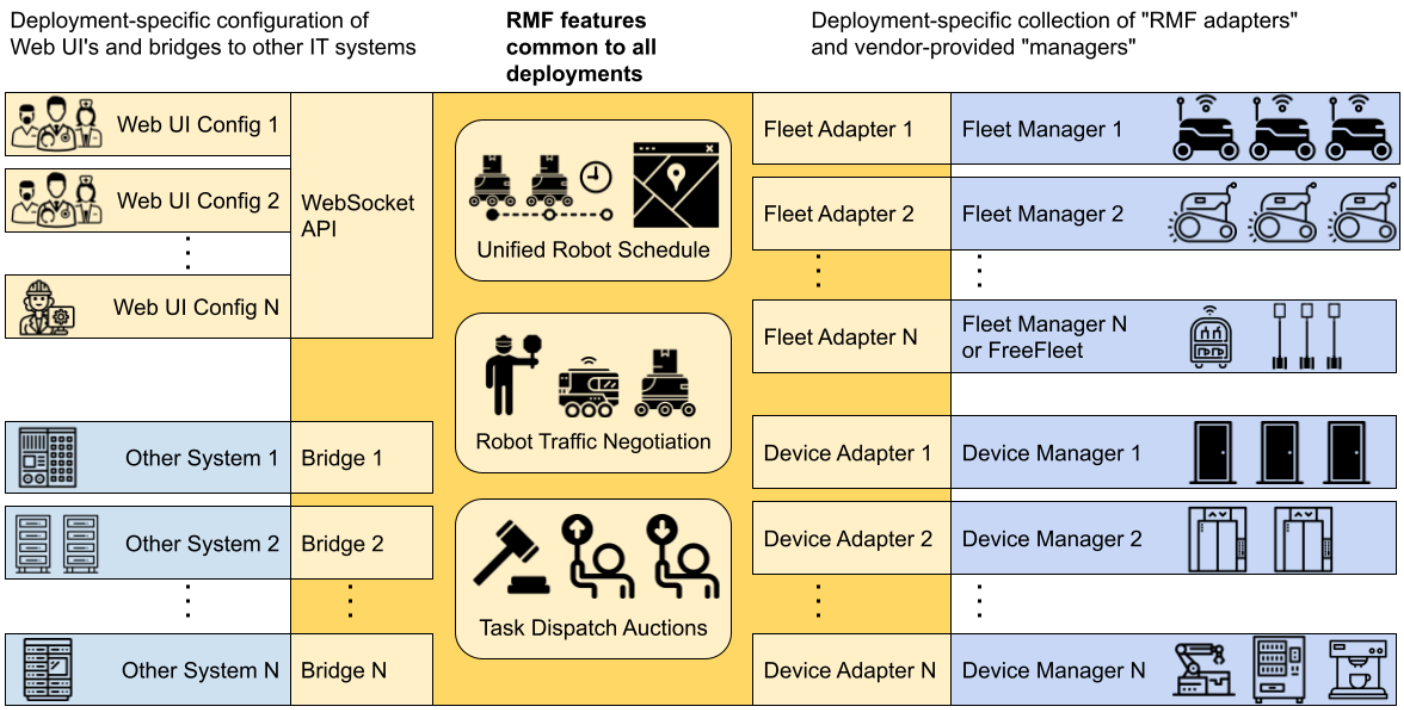
We would like to acknowledge the Singapore government for their vision and support to start this ambitious research and development project, "*Development of Standardised Robotics Middleware Framework - RMF detailed design and common services, large-scale virtual test farm infrastructure, and simulation modelling*". The project is supported by the Ministry of Health (MOH) and National Robotics Program (NRP).

Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s) and do not reflect the views of the NR2PO and MOH.

So what is RMF?

RMF is a collection of reusable, scalable libraries and tools building on top of ROS 2 that enable the interoperability of heterogeneous fleets of any type of robotic systems. RMF utilizes standardized communication protocols to infrastructure, environments and automation where robots are deployed to optimize the use of critical resources (i.e. robots, lifts, doors, passageways, etc). It adds intelligence to the system through resource allocation and by preventing conflicts over shared resources through the RMF Core which will be described in detail later in this book.

RMF is flexible and robust enough to operate over virtually any communications layer and integrate with any number of IOT devices. The architecture of RMF is designed in such a way to allow scalability as the level of automation in an environment increases. There are various ways for systems and users to interact with RMF via APIs and customizable user interfaces. Once deployed in an environment, RMF will save costs by allowing resources to be shared and integrations to be minimized. It is what robotic developers and robot customers have been looking for. In a nutshell, here is RMF:



How does RMF make the magic happen?

We will explore each of these functional areas in more detail in later chapters of this book, but for now we'd like to also introduce some of the other utilities helpful when developing and integrating with RMF.

RMF Demos

The demos are demonstrations of the capabilities of RMF in various environments. This repository serves as a starting point for working and integrating with RMF.

Traffic Editor

Traffic Editor is a GUI for creating and annotating floorplans for use in RMF. Through Traffic Editor you are able to create traffic patterns for use in RMF and introduce simulation models to enhance your virtual simulation environments. The `.yaml` files can be easily exported for use in Gazebo.

Free Fleet

Free Fleet is an open-source robot fleet management system for robot developers who do not have their own fleet manager or who would prefer to use and contribute to an open-source fleet management utility.

RMF Schedule Visualizer

This visualizer is an rviz-based `rmf_core` visualizer and control panel. It is intended to be a functional tool for RMF developers.

RMF Web UI

rmf-web is a configurable web application that provides overall visualization and control over the RoMi-H system. The dashboard is by design more "operator-friendly" compared to the previously mentioned schedule visualizer.

RMF Simulation

rmf_simulation contains the simulation plugins to simulate RMF. Plugins are available in `gazebo` and `ignition`.

Simulation Assets

The open-source and freely distributable simulation assets are created and shared to accelerate simulation efforts.

For the latest instructions and updates please directly check the open-rmf/rmf repository.

Install ROS 2.

First, please follow the installation instructions for ROS 2. If you are on an Ubuntu 20.04 LTS machine (as recommended), here is the binary install page for ROS 2 Galactic on Ubuntu 20.04.

Setup Gazebo repositories

Setup your computer to accept Gazebo packages from packages.osrfoundation.org.

```
sudo apt update
sudo apt install -y wget
sudo sh -c 'echo "deb http://packages.osrfoundation.org/gazebo/ubuntu-stable
`lsb_release -cs` main" > /etc/apt/sources.list.d/gazebo-stable.list'
wget https://packages.osrfoundation.org/gazebo.key -O - | sudo apt-key add -
```

Binary install

OpenRMF binary packages are available for Ubuntu Focal 20.04 for the `Foxy`, `Galactic` and `Rolling` releases of ROS 2. Most OpenRMF packages have the prefix `rmf` on their name, therefore, you can find them by them by searching for the pattern `ros-<ro2distro>-rmf`

```
apt-cache search ros-<ro2distro>-rmf
```

RMF Demos

A good way to install the `rmf` set of packages in one go is to install the one of the main RMF Demos packages. This will pull all the rest of the OpenRMF packages as a dependency. The core of RMF demos is contained on the `rmf_demos` package. However, if you want to install it with simulation support, you should install the `rmf_demos_gz` or `rmf_demos_ign` package which come with gazebo or ignition support respectively. To install the ROS 2 release with gazebo support package, you would run:

```
sudo apt install ros-<ro2distro>-rmf-demos-gz
```

Building from sources

If you want to get the latest developments you might want to install from sources and compile OpenRMF yourself.

Additional Dependencies

Install all non-ROS dependencies of OpenRMF packages,

```
sudo apt update && sudo apt install \
  git cmake python3-vcstool curl \
  qt5-default \
  -y
python3 -m pip install flask-socketio
sudo apt-get install python3-colcon*
```

Install rosdep

`rosdep` helps install dependencies for ROS packages across various distros. It can be installed with:

```
sudo apt install python3-rosdep
sudo rosdep init
rosdep update
```

Download the source code

Setup a new ROS 2 workspace and pull in the demo repositories using `vcs`,

```
mkdir -p ~/rmf_ws/src
cd ~/rmf_ws
wget https://raw.githubusercontent.com/open-rmf/rmf/main/rmf.repos
vcs import src < rmf.repos
```

Ensure all ROS 2 prerequisites are fulfilled,

```
cd ~/rmf_ws
rosdep install --from-paths src --ignore-src --rosdistro <ro2distro> -y
```

Compiling Instructions

> NOTE: Due to newer changes in the source build, there might be conflicts and compilation errors with older header files installed by the binaries. Please remove the binary installations before building from source, using `sudo apt remove ros-<ro2distro>-rmf*`.

Compiling on `Ubuntu 20.04`:

```
cd ~/rmf_ws
source /opt/ros/<ro2distro>/setup.bash
colcon build --cmake-args -DCMAKE_BUILD_TYPE=Release
```

> NOTE: The first time the build occurs, many simulation models will be downloaded from Ignition Fuel to populate the scene when the simulation is run. As a result, the first build can take a very long time depending on the server load and your Internet connection (for example, 60 minutes).

Run RMF Demos

Demonstrations of OpenRMF are shown in rmf_demos.

Docker Containers

Alternatively, you can run RMF Demos by using docker.

Pull docker image from `open-rmf/rmf` github registry (setup refer here).

```
docker pull ghcr.io/open-rmf/rmf/rmf_demos:latest
docker tag ghcr.io/open-rmf/rmf/rmf_demos:latest rmf:latest
```

Run it!

```
docker run -it --network host rmf:latest bash -c "export ROS_DOMAIN_ID=9; ros2
launch rmf_demos_gz office.launch.xml headless:=1"
```

This will run `rmf_demos` in headless mode. Open this link with a browser to start a task.

(Experimental) User can also run `rmf_demos` in "non-headless" graphical form, via rocker.

Roadmap

A near-term roadmap of the entire OpenRMF project (including and beyond `rmf_traffic`) can be found in the user manual here.

Integrating with RMF

Instructions on how to integrate your system with OpenRMF can be found here.

Open sourced fleet adapters

A number of commercial robots have been integrated with RMF and links to their adapters are available below.

- Gaussian Ecobots
- OTTO Motors (and robots running the Clearpath Autonomy stack)
- Mobile Industrial Robots: MiR
- Temi- the personal robot

Help us add to this list!

A helpful starting point for integrating your fleet with RMF is the fleet_adapter_template package.
